

LAPORAN PRAKTIKUM
PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 3: Blade Template Laravel



Disusun Oleh:

Nama : Mochammad Eyckies Bintang Satryaz

NIM : 707012500042

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 3 ini bertujuan agar mahasiswa dapat memahami dan menerapkan Blade Template pada framework Laravel. Secara lebih rinci, tujuan yang ingin dicapai pada praktikum ini adalah sebagai berikut.

1. Memahami konsep Blade Template sebagai template engine bawaan Laravel beserta proses kompilasinya menjadi kode PHP.
2. Menerapkan sintaks dasar Blade, seperti `{{ }}` untuk menampilkan data serta directive `@if`, `@elseif`, `@else`, dan `@foreach`.
3. Menerapkan fitur layouting menggunakan `@extends`, `@section`, dan `@yield` agar setiap halaman memiliki tampilan yang seragam.
4. Membuat komponen Blade custom (`<x-alert>`) dan partial (`@include`) sebagai bagian tampilan yang dapat digunakan berulang.
5. Membangun aplikasi prototipe pelaporan banjir bernama `LaporBanjir` menggunakan pola MVC (Model-View-Controller) tanpa database.
6. Menghubungkan setiap halaman dengan route bernama (named route) sehingga mudah dipanggil dari navigasi.

2. Alat dan Bahan

Praktikum ini dilaksanakan menggunakan perangkat dan perangkat lunak berikut.

2.1 Perangkat Keras

- Laptop/komputer dengan sistem operasi Windows.
- Prosesor minimal 1 GHz, RAM minimal 4 GB, dan ruang penyimpanan kosong minimal 2 GB.

2.2 Perangkat Lunak

Perangkat Lunak	Versi / Keterangan
XAMPP (Apache & MySQL)	Server lokal, PHP 8.2.12
Composer	2.10.3 (dependency manager PHP)
Laravel Framework	12.69.2
Visual Studio Code	Teks editor / IDE
Google Chrome	Browser untuk pengujian
Git	Kontrol versi dan pengumpulan tugas

3. Langkah-Langkah Praktikum

Pengerjaan Tugas Praktikum nomor 1 (aplikasi `LaporBanjir`) dilakukan secara bertahap mengikuti alur MVC, yaitu route, controller, lalu view. Berikut tahapan yang dikerjakan.

3.1 Membuat Proyek dan Route

Proyek Laravel dibuat terlebih dahulu, kemudian route didaftarkan pada file routes/web.php. Route diarahkan ke method pada LaporanBanjirController dan diberi nama agar dapat dipanggil menggunakan helper route().

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\LaporanBanjirController;

Route::get('/', [LaporanBanjirController::class, 'form']);

Route::get('/lapor-banjir', [LaporanBanjirController::class, 'form'])->name('lapor-banjir.form');
Route::post('/lapor-banjir/kirim', [LaporanBanjirController::class, 'kirim'])->name('lapor-banjir.kirim');
Route::get('/lapor-banjir/daftar', [LaporanBanjirController::class, 'daftar'])->name('lapor-banjir.daftar');
```

Terdapat tiga route utama: form pelaporan (GET), pengiriman laporan (POST), dan daftar laporan (GET). Route POST digunakan karena data form dikirim melalui method POST.

3.2 Membuat Controller

Controller dibuat dengan perintah php artisan make:controller LaporanBanjirController. Controller berperan menyiapkan data dan mengembalikan view. Method kirim() memvalidasi input form, sedangkan method daftar() menyiapkan data laporan berupa array (tanpa database).

```
public function form()
{
    return view('lapor-banjir.form');
}

public function kirim(Request $request)
{
    $validated = $request->validate([
        'nama_pelapor' => ['required', 'string', 'max:100'],
        'lokasi' => ['required', 'string', 'max:150'],
        'tinggi_genangan' => ['required', 'numeric', 'min:1'],
    ], [
        'nama_pelapor.required' => 'Nama pelapor wajib diisi.',
        'lokasi.required' => 'Lokasi kejadian wajib diisi.',
        'tinggi_genangan.numeric' => 'Tinggi genangan harus berupa angka.',
        'tinggi_genangan.min' => 'Tinggi genangan minimal 1 cm.',
    ]);

    return view('lapor-banjir.konfirmasi', $validated);
}
```

```

}

public function daftar()
{
    $laporan = [
        ['nama_pelapor' => 'Dina Rahma', 'lokasi' => 'Baleendah, Desa Andir', 'tinggi_genangan' => 25],
        ['nama_pelapor' => 'Rizky Pratama', 'lokasi' => 'Dayeuhkolot, Citeureup', 'tinggi_genangan' =>
55],
        ['nama_pelapor' => 'Siti Nurjanah', 'lokasi' => 'Bojongsoang, Tegalluar', 'tinggi_genangan' =>
82],
        ['nama_pelapor' => 'Agus Hermawan', 'lokasi' => 'Majalaya, Padamulya', 'tinggi_genangan' =>
30],
    ];

    return view('lapor-banjir.daftar', compact('laporan'));
}

```

3.3 Membuat Layout Utama

Layout utama dibuat pada `resources/views/layouts/app.blade.php`. Layout ini memuat bagian yang sama di setiap halaman, yaitu header berisi nama sistem, menu navigasi berbasis named route, footer, dan CSS. Bagian yang berubah ditandai dengan `@yield`, sedangkan halaman turunan mengisinya lewat `@section` dan `@extends`.

```

<title>@yield('title', 'Praktikum Modul 3')</title>
...
<h1>@yield('app_name', 'Praktikum Modul 3')</h1>
<p>@yield('tagline', 'Aplikasi web praktikum')</p>

<nav aria-label="Navigasi utama">
    <a href="{{ route('lapor-banjir.form') }}"
        @class(['active' => request()->routeIs('lapor-banjir.form')])>Form Banjir</a>
    <a href="{{ route('lapor-banjir.daftar') }}"
        @class(['active' => request()->routeIs('lapor-banjir.daftar')])>Daftar Laporan</a>
</nav>

@yield('content')

```

Directive `@class()` digunakan untuk menandai menu yang sedang aktif berdasarkan route yang dibuka, sehingga navigasi terasa dinamis.

3.4 Membuat Halaman Form Pelaporan

Halaman form mewarisi layout utama dan berisi form yang dikirim dengan method POST ke route lapor-banjir.kirim. Directive @csrf ditambahkan sebagai pengaman, old() mempertahankan isi input saat validasi gagal, dan @error menampilkan pesan kesalahan.

```
@extends('layouts.app')

@section('title', 'LaporBanjir - Form Pelaporan')
@section('app_name', 'LaporBanjir')
@section('tagline', 'Sistem pelaporan banjir BPBD Kabupaten Bandung')

@section('content')
<form action="{{ route('lapor-banjir.kirim') }}" method="post">
    @csrf

    <div class="field">
        <label for="nama_pelapor">Nama Pelapor</label>
        <input type="text" id="nama_pelapor" name="nama_pelapor"
            value="{{ old('nama_pelapor') }}" placeholder="Contoh: Eyckies Bintang">
        @error('nama_pelapor')
            <div class="error">{{ $message }}</div>
        @enderror
    </div>

    <div class="field">
        <label for="lokasi">Lokasi Kejadian</label>
        <input type="text" id="lokasi" name="lokasi"
            value="{{ old('lokasi') }}" placeholder="Contoh: Kecamatan Baleendah, Desa Andir">
        @error('lokasi')
            <div class="error">{{ $message }}</div>
        @enderror
    </div>

    <div class="field">
        <label for="tinggi_genangan">Tinggi Genangan Air (cm)</label>
        <input type="number" id="tinggi_genangan" name="tinggi_genangan"
            value="{{ old('tinggi_genangan') }}" min="1" placeholder="Contoh: 45">
        @error('tinggi_genangan')
            <div class="error">{{ $message }}</div>
        @enderror
    </div>
</form>
```

```

        <button type="submit">Kirim Laporan</button>
    </form>
@endsection

```

3.5 Membuat Halaman Konfirmasi dan Komponen Alert

Komponen alert dibuat pada `resources/views/components/alert.blade.php`. File di dalam folder `components` otomatis dikenali sebagai komponen `<x-alert>`. Komponen ini menerima prop `type` dan konten melalui slot, sehingga dapat dipakai ulang di banyak halaman.

```

@props(['type' => 'success'])

<div class="alert {{ $type }}" role="alert">
    {{ $slot }}
</div>

```

Halaman konfirmasi menampilkan pesan keberhasilan menggunakan komponen `<x-alert>` dan menampilkan kembali data yang telah dikirim dari form.

```

@extends('layouts.app')

@section('title', 'LaporBanjir - Konfirmasi')
@section('content')
    <x-alert>
        Laporan berhasil dikirim. Data berikut sudah diterima oleh sistem prototipe LaporBanjir.
    </x-alert>

    <div class="summary">
        <div class="row">
            <div class="label">Nama Pelapor</div>
            <div class="value">{{ $nama_pelapor }}</div>
        </div>
        <div class="row">
            <div class="label">Lokasi Kejadian</div>
            <div class="value">{{ $lokasi }}</div>
        </div>
        <div class="row">
            <div class="label">Tinggi Genangan</div>
            <div class="value">{{ $tinggi_genangan }} cm</div>
        </div>
    </div>
@endsection

```

3.6 Membuat Halaman Daftar Laporan dan Partial Kartu

Halaman daftar menampilkan data laporan menggunakan `@forelse`. Setiap laporan dirender melalui partial `resources/views/partials/laporan-card.blade.php` yang dipanggil dengan `@include`. Jika data kosong, blok `@empty` menampilkan pesan alternatif. Status genangan ditentukan dengan `@if/@elseif/@else`.

```
{{-- daftar.blade.php --}}
@extends('layouts.app')

@section('content')
<section class="grid">
    @forelse ($laporan as $item)
        @include('partials.laporan-card', ['laporan' => $item])
    @empty
        <div class="panel">Belum ada laporan banjir.</div>
    @endforelse
</section>
@endsection

{{-- partials/laporan-card.blade.php --}}
<article class="card">
    <div class="card-body">
        @if ($laporan['tinggi_genangan'] < 30)
            <span class="status waspada">Waspada</span>
        @elseif ($laporan['tinggi_genangan'] <= 70)
            <span class="status siaga">Siaga</span>
        @else
            <span class="status awas">Awat</span>
        @endif
        <h3>{{ $laporan['lokasi'] }}</h3>
        <p>Pelapor: <strong>{{ $laporan['nama_pelapor'] }}</strong></p>
        <p>Tinggi genangan: <strong>{{ $laporan['tinggi_genangan'] }} cm</strong></p>
    </div>
</article>
```

Aturan status genangan pada partial: tinggi kurang dari 30 cm berstatus Waspada, 30 sampai 70 cm berstatus Siaga, dan lebih dari 70 cm berstatus Awat.

3.7 Menjalankan Server dan Pengujian

Aplikasi dijalankan dengan perintah php artisan serve, lalu diuji melalui browser pada alamat yang tersedia.

```
php artisan serve
```

```
# Halaman form pelaporan
```

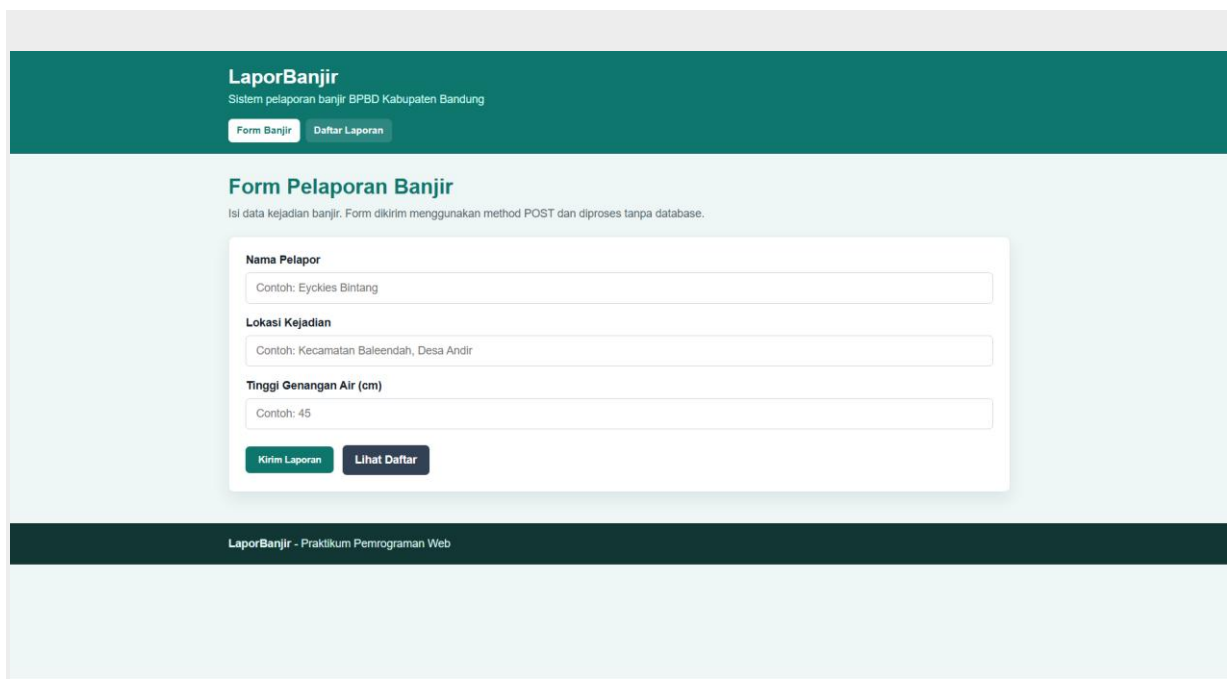
```
http://127.0.0.1:8000/lapor-banjir
```

```
# Halaman daftar laporan
```

```
http://127.0.0.1:8000/lapor-banjir/daftar
```

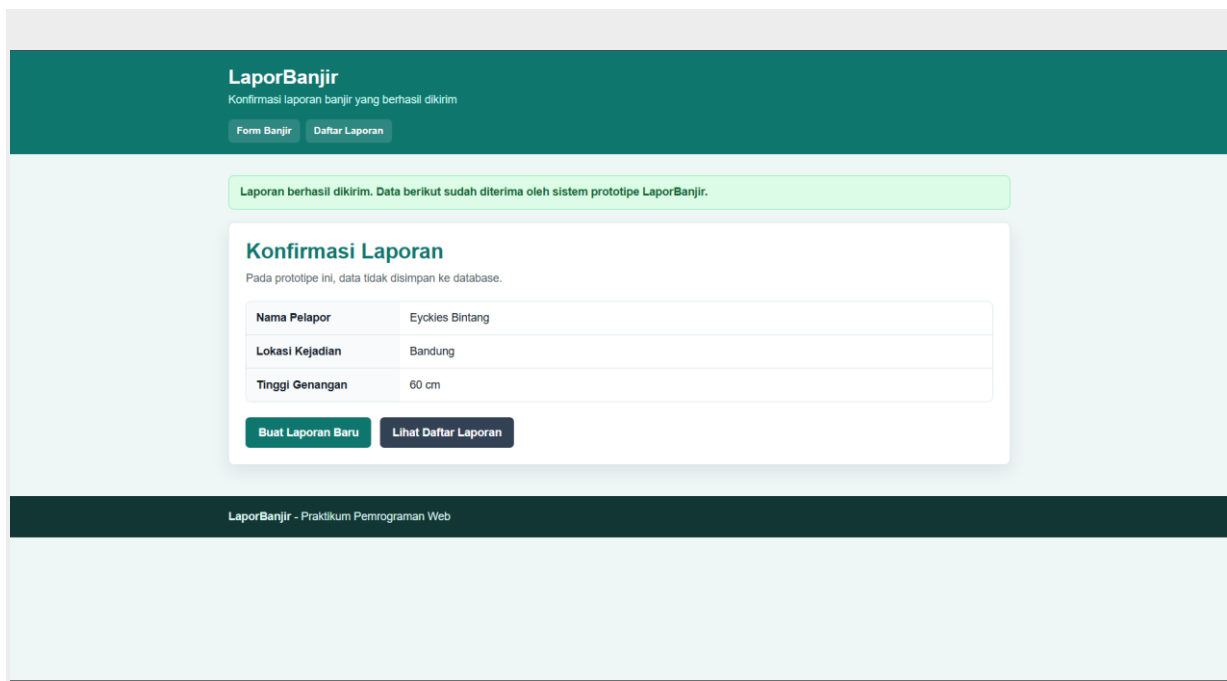
4. Hasil dan Dokumentasi Program

Seluruh halaman berhasil dijalankan dengan tampilan seragam karena mewarisi satu layout utama. Berikut dokumentasi hasil aplikasi LaporanBanjir.



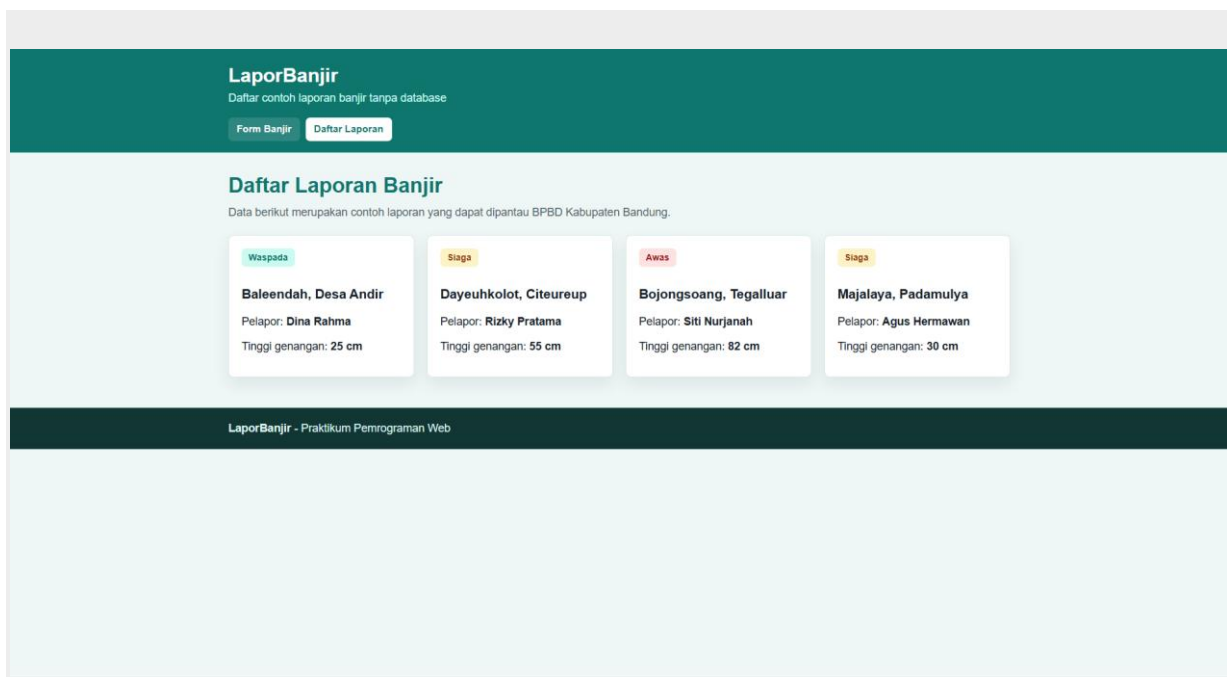
Gambar 1. Halaman Form Pelaporan Banjir

Gambar 1 memperlihatkan halaman form pelaporan yang berisi tiga input, yaitu nama pelapor, lokasi kejadian, dan tinggi genangan air. Form dikirim menggunakan method POST ke route lapor-banjir.kirim.



Gambar 2. Halaman Konfirmasi dengan Komponen Alert

Gambar 2 memperlihatkan pesan keberhasilan laporan yang ditampilkan melalui komponen `<x-alert>`, diikuti ringkasan data yang baru saja dikirim.



Gambar 3. Halaman Daftar Laporan dengan Status Genangan

Gambar 3 memperlihatkan daftar laporan yang dirender menggunakan @forelse dan partial kartu. Setiap kartu menampilkan status genangan (Waspada, Siaga, atau Awas) yang ditentukan melalui @if/@elseif/@else.

5. Kesimpulan

Berdasarkan praktikum Modul 3 ini, dapat disimpulkan bahwa Blade Template mempermudah penyusunan tampilan Laravel karena HTML dapat digabungkan dengan sintaks ringkas seperti {{ }}, @if, @foreach, dan @forelse, serta dikompilasi otomatis menjadi kode PHP.

Fitur layouting menggunakan @extends, @section, dan @yield membuat seluruh halaman LaporanBanjir memiliki header, navigasi, dan footer yang seragam sehingga cukup diubah pada satu file layout utama. Komponen <x-alert> dan partial @include juga terbukti membuat kode lebih rapi dan dapat digunakan berulang, sehingga pemeliharaan tampilan menjadi lebih mudah.

Aplikasi LaporanBanjir berhasil dibangun dengan pola MVC tanpa database, di mana data cukup ditulis dalam bentuk array pada Controller. Seluruh halaman form, konfirmasi, dan daftar laporan dapat diakses melalui named route, dan status genangan berhasil ditentukan sesuai ketentuan, yaitu Waspada (< 30 cm), Siaga (30-70 cm), dan Awas (> 70 cm).

6. Link GitHub

Kode program lengkap praktikum ini dapat diakses pada repositori berikut.

<https://github.com/kies-kis/PABW>